

AI 데이터 분석 · 12일차

모듈과 경로

직접 만들기보다 가져다 쓰기

`import · from · as · math · random · datetime · os · 경로 · getcwd · listdir · join · exists`

교안 08_01(모듈과 경로 다루기) 전 범위 + 08_02(파일 입출력) 도입부

처음으로 실제 설비 데이터 파일을 만나는 날

2026-08-05 AI 데이터 분석 교육 정리 (12일차)

학습자 Minho · 국립금오공과대학교 컴퓨터공학부

이 책을 읽는 법

어제까지는 **내가 함수를 만드는** 법을 배웠습니다. 오늘은 방향이 반대입니다 — **남이 이미 만들어 둔 것을 가져다 쓰는** 법입니다. 교안 표현으로 "직접 만들기보다 가져다 쓰기"죠. 그리고 후반부에는 처음으로 **실제 설비 데이터 파일**을 다루기 시작합니다.

모듈 import · from · as **내장함수** math · random · datetime **경로** 절대 · 상대 · 작업 디렉토리 **메서드** os.getcwd · listdir · join · exists **파일** open · with

연결

어제 배운 함수가 오늘 바로 쓰입니다. `math.sqrt(16)`은 누군가 **def**로 만들어 둔 함수를 호출하는 것이고, `os.path.join("Remind", "함수.pdf")`는 위치 인자 두 개를 넘기는 호출입니다. 강사님이 어제 "pandas는 사실상 함수 덩어리"라고 하신 그 감각이 오늘부터 실제로 필요해집니다.

목차

	장 / STEP	무엇을 배우나
CHAPTER 0	오늘의 지도 — 가져다 쓰기	오늘의 실데이터 · 소성가공 프레스
STEP 1	모듈이란 무엇인가	모듈 · 패키지 · 라이브러리
STEP 2	import 세 가지 방식	import · from import · as · 별표 회피
STEP 3	표준 라이브러리 5종	math · random · datetime · os · csv
STEP 4	경로 — 절대 · 상대 · 작업 디렉토리	파일을 못 찾는 진짜 이유
STEP 5	os 모듈로 경로 다루기	getcwd · listdir · join · exists
STEP 6	파일 입출력 시작	open · 모드 r · w · a · 인코딩 · with
STEP 7	내 코드 점검 — 수업 코드 2개	작업 디렉토리에 걸린 코드
STEP 8	실습 점검 — 기본 4 · 선택 2	버그 4건 · 재귀 대신 요구사항대로
부록 A · B · C	치트시트 · 오류 사전 · 예고	한 장 요약 · 함정 모음

오늘의 지도 — 가져다 쓰기

같은 코드를 매번 다시 짜지 않는다

어제 함수를 배우면서 "복사·붙여넣기를 두 번 이상 하면 신호"라고 했습니다. 오늘은 그 원칙을 파일 단위로 확장합니다 — **한 파일에서 만든 함수를 다른 파일에서도 쓰려면?** 그 답이 모듈입니다.

이유	뜻	실제로는
재사용	한 번 만든 코드를 다른 파일에서도 쓴다	복사·붙여넣기 없이 import 한 줄
관리	길어진 코드를 기능별 파일로 나눈다	찾기 쉽고 고치기 쉬워진다
검증	전 세계 개발자가 검증한 코드를 그대로 쓴다	직접 만드는 것보다 안전하다

개념

세 번째가 오늘 가장 중요합니다. 제공근을 구하는 코드를 직접 짜면 며칠이 걸리고 정확도도 장담 못 합니다. 그런데 `math.sqrt(16)` 한 줄이면 수십 년간 전 세계가 검증한 계산을 그대로 씁니다. 강사님도 "원리를 알아야 활용법을 이해한다"고 하셨지만, 그 다음 단계는 **잘 만들어진 것을 알아보고 가져다 쓰는 감각**입니다.

오늘의 실데이터 — 소성가공 프레스

교안 첫 장에 오늘부터 다룰 데이터가 소개됩니다. 지금까지 실습에서 쓰던 가짜 센서값이 아니라 **실제 설비 데이터**입니다.

항목	내용
공정	소성가공 프레스 — 금속을 강한 힘으로 눌러 형태를 만드는 설비
기록	작동 중 진동과 전류를 지속적으로 기록
정상	진동·전류가 일정 범위에 머문다 → 상태 0
이상	금형 마모나 이물이 끼면 값이 튜다 → 상태 1
파일	<code>data/08_press.csv</code>

08_press.csv — 첫 줄이 헤더, 그 아래가 기록

설비ID, 시각, 진동X, 진동Y, 전류, 상태

PRESS-01, 2022-07-12 00:00:00.019, 0.117, -0.1764, 192.3387, 0

PRESS-02, 2022-07-12 00:08:27.624, 0.242, 0.1, -51.347, 0

PRESS-09, 2022-07-12 01:16:55.828, 0.0773, 0.0883, 108.258, 0

PRESS-10, 2022-07-17 10:51:07.943, 0.2937, -0.5406, 21.4577, 1

PRESS-11, 2022-07-17 10:52:40.072, 0.1102, -0.1504, -85.8307, 1

설비 적용

마지막 열 **상태**를 보세요. 위쪽 PRESS-01~09는 0(정상), 아래 PRESS-10~11은 1(이상)입니다. 그리고 이상 행의 **진동X가 0.29, 진동Y가 -0.54**로 정상 구간(0.01~0.24)보다 확연히 큼니다. 앞으로 우리가 할 일이 바로 이것입니다 — **이 파일을 읽어 들여, 값이 튀는 행을 골라내고, 상태 1을 예측하는 것.** 오늘은 그 첫 단계인 "파일을 찾아 여는 법"을 배웁니다.

오늘의 흐름



한 문장으로 붙이면 — **필요한 도구를 가져와서, 파일이 어디 있는지 찾아, 열어 본다.** 오늘은 여기까지이고, 실제로 읽고 쓰는 것은 내일입니다.

STEP 1

모듈이란 무엇인가

파일 · 폴더 · 통칭

교안의 정의는 한 줄입니다 — "모듈은 코드가 담긴 .py 파일 하나." 어제 우리가 만든 17_function2.py도 이미 모듈입니다. 다른 파일에서 import하면 그 안의 함수를 가져다 쓸 수 있죠.

모듈 = 함수와 변수가 모인 도구 상자

```
# math.py 안에는 이런 것들이 들어 있다 (개념도)
def sqrt(x): ...
def ceil(x): ...
def floor(x): ...
pi = 3.14159...
e = 2.71828...
```

비유

교안이 든 비유는 **도구 상자**입니다. 망치와 드라이버가 든 상자처럼 — math 모듈은 수학 도구가 담긴 상자, os 모듈은 폴더·파일 도구가 담긴 상자입니다. 우리는 상자를 통째로 가져와 필요한 도구만 꺼내 씁니다.

모듈 모듈 · 패키지 · 라이브러리 파일 · 폴더 · 통칭

정의 셋 다 "가져와 쓴다"는 점은 같고, **단위가 다릅니다**. 모듈은 파일 하나, 패키지는 폴더, 라이브러리는 그 둘을 아우르는 통칭입니다.

왜 쓰나 용어가 세 개나 되는 이유는 규모 때문입니다. math는 파일 하나로 충분하지만, pandas 같은 큰 도구는 파일 수백 개를 폴더로 묶어야 합니다. 실무 대화에서는 셋을 엄밀히 구분하지 않고 "**라이브러리**"로 뭉뚱그려 부르는 경우가 많습니다.

응용 우리가 이미 만든 파일도 모듈로 쓸 수 있습니다. 파일 이름이 곧 모듈 이름이 되므로, **파일 이름을 math.py**처럼 표준 라이브러리와 같게 지으면 충돌이 납니다. 어제 배운 "**내장 함수 이름을 변수로 쓰지 말라**"와 같은 원칙이 **파일 이름에도 적용되는 셈입니다**.

```
# my_tools.py (내가 만든 모듈)
def calc_average(values):
    """숫자 목록의 평균을 돌려준다."""
    return sum(values) / len(values)

# main.py - 같은 폴더에서
import my_tools
print(my_tools.calc_average([78, 92, 85]))

▶ 85.0
```

개념

.py 확장자는 **빼고** 씁니다. `import my_tools.py`가 아니라 `import my_tools`입니다. 파이썬이 알아서 .py를 붙여 찾습니다.

세 단위 비교

용어	단위	예	비유
모듈	.py 파일 하나	<code>math · random · os</code>	도구 상자 하나
패키지	관련 모듈을 모은 폴더	<code>numpy · pandas</code>	상자를 넣은 공구함
라이브러리	둘을 아우르는 통칭	표준 라이브러리 · 외부 라이브러리	창고 전체

연결

어제 강사님이 "**pandas는 사실상 함수 덩어리**"라고 하셨는데, 정확히 말하면 pandas는 **패키지**입니다 — 수백 개 모듈이 폴더로 묶여 있고, 그 안에 우리가 쓸 함수들이 들어 있습니다. 오늘 배우는 `import`가 그것을 여는 열쇠입니다.

STEP 2

import 세 가지 방식

통째로 · 일부만 · 별칭으로

키워드 import 모듈명 통째로 가져오기

정의 모듈 전체를 가져옵니다. 사용할 때는 모듈명.기능명() 형태로 점을 찍어 꺼내 씁니다.

형태 import 모듈명 모듈명.기능명()

```
import math

result = math.sqrt(16)
print(result)
print(math.pi)
```

▶ 4.0
▶ 3.141592653589793

왜 쓰나 출처가 명확한 것이 가장 큰 장점입니다. math.sqrt라고 쓰여 있으면 이 기능이 어디서 왔는지 코드만 봐도 압니다. 대신 매번 모듈명을 붙여야 해서 길어집니다.

응용 import는 파일 맨 위에 한 줄씩 모아 적는 것이 관례(PEP 8)입니다. 이 코드가 어떤 도구를 쓰는지 한눈에 보이기 때문입니다. 실제로 파일 중간에서 import해도 동작은 하지만, 읽는 사람이 도구 목록을 파악하기 어려워집니다.

```
# ✓ 파일 맨 위에 모아 적는다
import math
import random
import os

# x 필요할 때마다 흩어 놓는다 (동작은 하지만 읽기 나쁨)
print("시작")
import math
print(math.sqrt(16))
```

개념

점 표기법 math.sqrt는 어제 배운 3초 판별법 그대로입니다 — 점이 있고 괄호가 붙으면 메서드처럼 보이지만, math는 객체가 아니라 모듈이므로 정확히는 **모듈 안의 함수**입니다. 실무에서는 굳이 구분하지 않고 "math의 sqrt 함수"라고 부릅니다.

키워드 from 모듈 import 기능 일부만 가져오기

정의 모듈에서 **필요한 기능만** 콕 집어 가져옵니다. 사용할 때 모듈명을 붙이지 않습니다.

형태 from 모듈명 import 기능명 기능명()

```
from math import sqrt

result = sqrt(16)      # math. 없이 바로
print(result)

from math import sqrt, pi, ceil    # 여러 개도 가능
print(pi, ceil(3.2))
```

▶ 4.0
▶ 3.141592653589793 4

왜 쓰나 코드가 간결해집니다. 같은 기능을 수십 번 쓸 때 매번 math.를 붙이지 않아도 됩니다.

응용 단점은 **출처가 가려진**다는 것입니다. 코드 중간에 sqrt(16)만 보이면 이게 어디서 온 기능인지 파일 맨 위까지 올라가 봐야 합니다. 그리고 내가 만든 함수와 이름이 겹치면 조용히 덮어써집니다 — 어제 배운 "같은 이름 함수 재정의" 문제와 같습니다.

```
from math import sqrt

def sqrt(x):          # ← 내가 만든 함수가 덮어쓴다
    return "내 함수"

print(sqrt(16))       # math의 sqrt가 아니다
```

▶ 내 함수

자주 하는 실수

그래서 실무에서는 **자주 쓰는 소수의 기능만** from import로 가져오고, 나머지는 import 모듈명을 쓰는 절충이 일반적입니다.

키워드 import 모듈 as 별칭 짧은 별명 붙이기

정의 긴 모듈명에 짧은 별명을 붙여 씁니다. **출처는 유지하면서 길이는 줄이는** 절충안입니다.

형태 import 모듈명 as 별칭 별칭.기능명()


```
import math as mt

result = mt.sqrt(16)
print(result)

import datetime as dt
now = dt.datetime.now()
print(type(now).__name__)
```

▶ 4.0
▶ datetime

왜 쓰나 데이터 분석 쪽에는 **전 세계가 따르는 공통 약속**이 있습니다. 아무 별명이나 붙이는 게 아니라 관례를 따르는 것이 중요합니다.

응용 아래 세 줄은 데이터 분석 코드라면 거의 반드시 등장합니다. 인터넷의 예제 코드도 전부 이 별칭을 쓰므로, 다른 이름을 붙이면 오히려 읽기 어려워집니다.

```
import numpy as np          # 수치 계산
import pandas as pd         # 표 데이터
import matplotlib.pyplot as plt # 그래프

# 앞으로 보게 될 코드는 대부분 이 형태
# df = pd.read_csv("data/08_press.csv")
# plt.plot(df["진동X"])
```

주의

`import matplotlib.pyplot as plt`는 **패키지 안의 모듈**을 가져오는 형태입니다. `matplotlib`이 패키지(폴더), `pyplot`이 그 안의 모듈(파일)이죠. STEP 1에서 배운 구분이 여기서 실제로 보입니다.

세 방식 비교

구분	import 모듈명	from ... import	import ... as
작성	<code>import math</code>	<code>from math import sqrt</code>	<code>import math as mt</code>
사용	<code>math.sqrt(16)</code>	<code>sqrt(16)</code>	<code>mt.sqrt(16)</code>
장점	출처 명확	코드 간결	둘의 절충
단점	매번 모듈명	출처가 가려짐	별칭을 외워야 함

자주 하는 실수

네 번째 방식으로 `from math import *`(별표)가 있습니다. 모든 기능을 한꺼번에 가져오는 건데, 교안이 "실무 회피"로 표시한 방식입니다. 이유는 두 가지 — ① `sqrt`, `pi`, `ceil` 등 수십 개가 한꺼번에 들어와 내 변수와 이름이 충돌할 수 있고 ② 코드 어디에도 출처가 남지 않아 어디서 온 기능인지 알 수 없기 때문입니다. 편해 보이지만 나중에 반드시 대가를 치릅니다.

내가 만든 파일도 모듈이 된다

어제 만든 함수들을 다른 파일에서 쓰려면 어떻게 할까요? 답은 이미 배웠습니다 — 같은 폴더에 두고 `import` 하면 됩니다. 파일 이름이 곧 모듈 이름이 되고, `.py`는 뺍니다.

만드는 쪽

```
# my_tools.py - 내가 만든 모듈

def calc_average(values):
    """숫자 목록의 평균을 돌려준다."""
    return sum(values) / len(values)

def judge(avg, limit=90.0):
    """평균과 기준값을 받아 상태를 돌려준다."""
    if avg > limit:
        return "위험"
    return "정상"
```

쓰는 쪽

```
# main.py - 같은 폴더에서
import my_tools

avg = my_tools.calc_average([78, 92, 85])
print(avg, my_tools.judge(avg))

# 또는 필요한 것만
from my_tools import calc_average, judge

avg = calc_average([78, 92, 85])
print(avg, judge(avg))
```

▶ 85.0 정상

▶ 85.0 정상

개념

어제 배운 **함수를 여러 파일에서 재사용**하는 방법이 바로 이것입니다. CHAPTER 0에서 말한 "재사용"이 구체적으로 무엇인지 여기서 드러냅니다 — 복사·붙여넣기 대신 `import` 한 줄. 그리고 `math.os`도 결국은 누군가가 이렇게 만들어 둔 `.py` 파일일 뿐입니다.

자주 하는 실수

파일 이름을 지을 때 **표준 라이브러리와 겹치지 않게** 해야 합니다. 내 파일을 `math.py`나 `random.py`로 지으면, `import math`가 표준 라이브러리 대신 **내 파일을 가져옵니다**. 그러면 `math.sqrt`가 없다는 오류가 나는데 원인을 찾기가 매우 어렵습니다. 어제 배운 "내장 함수 이름을 변수로 쓰지 말라"가 **파일 이름에도 그대로 적용**되는 셈입니다.

STEP 3

표준 라이브러리 5종

설치 없이 바로 쓰는 도구들

표준 라이브러리는 파이썬을 설치하면 함께 들어오는 모듈 묶음입니다. 수백 개가 있지만 데이터 분석 입문에서 만날 것은 다섯 개입니다. 지금은 각자 무슨 일을 하는지 **큰 그림만** 잡으면 됩니다.

모듈	하는 일	활용 예	오늘 배움
math	수학 계산	제곱근 · 원주율 — 통계 계산	○
random	무작위 처리	가짜 센서값 — 실데이터 없을 때 연습용	○
datetime	날짜 · 시간	점검 시각 — 언제 했는지 기록	○
os	폴더 · 파일	파일 목록 — 데이터 폴더 탐색	○
CSV	표 데이터	센서 CSV 읽기 · 쓰기	내일

모듈 math 수학 계산 도구 상자

정의 제곱근·올림·내림·원주율 등 한 단계 전문적인 수학 계산을 제공합니다.

```
import math

print(math.sqrt(16))      # 제곱근
print(math.ceil(3.2))     # 올림
print(math.floor(3.8))    # 내림
print(math.pi)           # 원주율
print(round(3.14159, 2))  # round는 내장 함수 — import 불필요

▶ 4.0
▶ 4
▶ 3
▶ 3.141592653589793
▶ 3.14
```

왜 쓰나 통계 계산에서 자주 쓰입니다. 표준편차를 직접 구하려면 제곱근이 필요하고, 구간을 나눌 때 올림·내림이 필요합니다.

응용 설비 데이터에서 진동 크기를 구할 때가 좋은 예입니다. 진동X와 진동Y가 따로 기록되어 있으면 **두 축을 합친 실제 크기**는 피타고라스 정리로 구합니다.

```
import math
```

```
vib_x, vib_y = 0.2937, -0.5406      # PRESS-10 (이상 상태)
size = math.sqrt(vib_x**2 + vib_y**2)
print(f"진동 크기: {round(size, 4)}")
```

```
vib_x, vib_y = 0.117, -0.1764      # PRESS-01 (정상 상태)
size = math.sqrt(vib_x**2 + vib_y**2)
print(f"진동 크기: {round(size, 4)}")
```

▶ 진동 크기: 0.6152

▶ 진동 크기: 0.2117

개념

`round()`는 **내장 함수**라 `import` 없이 씁니다. `math.sqrt()`는 모듈 안의 함수라 `import math`가 필요하고요. **얼마나 기본적인 기능인가**로 갈립니다 — `print`·`len`·`round`·`sum`·`open`은 내장, 그보다 전문적인 것은 모듈에 있습니다.

모듈 random 무작위 처리

정의 무작위 숫자를 만들거나 목록에서 하나를 골라 줍니다. 어제 `random.choice()`로 이미 써 봤습니다.

```
import random
```

```
print(random.randint(1, 100))      # 1~100 정수 (100 포함)
print(random.choice(["A", "B", "C"]))
print(round(random.random(), 4))    # 0.0~1.0 실수
```

▶ 47 (실행할 때마다 달라짐)

▶ B

▶ 0.7321

왜 쓰나 실제 데이터가 없을 때 연습용 가짜 값을 만드는 데 씁니다. 오늘 실습 2가 정확히 그 용도입니다.

응용 `randint(a, b)`는 양쪽 끝을 모두 포함합니다. `range(1, 100)`이 99까지만 것과 다르니 주의해야 합니다.

실행할 때마다 값이 달라지는데, 같은 값을 재현하고 싶으면 `random.seed()`로 씨앗을 고정합니다.

```
import random

random.seed(42)          # 씨앗 고정 → 항상 같은 결과
print([random.randint(1, 100) for _ in range(5)])

random.seed(42)          # 다시 같은 씨앗
print([random.randint(1, 100) for _ in range(5)])  # 같은 값
```

```
▶ [82, 15, 4, 95, 36]
▶ [82, 15, 4, 95, 36]
```

주의

`random.seed()`는 **버그를 재현할 때** 유용합니다. 무작위 데이터로 테스트하다 오류가 났는데 다시 실행하면 안 나는 상황이 있죠. 씨앗을 고정해 두면 같은 데이터로 몇 번이든 재현할 수 있습니다.

모듈 datetime 날짜와 시간

정의 현재 시각을 가져오거나 날짜를 계산합니다. **모듈 안에 같은 이름의 클래스**가 있어 표기가 조금 헷갈립니다.

형태 `import datetime datetime.datetime.now()`

```
import datetime

now = datetime.datetime.now()  # 모듈.클래스.함수
print(now)
print(type(now))

import datetime as dt         # 별칭을 쓰면 조금 짧아진다
print(dt.datetime.now())
```

```
▶ 2026-08-05 11:19:45.776780
▶ <class 'datetime.datetime'>
▶ 2026-08-05 11:19:45.776912
```

왜 쓰나 점검을 언제 했는지 기록할 때 씁니다. 설비 데이터는 시각이 생명이라 앞으로 계속 나옵니다. `08_press.csv`의 두 번째 열도 시각이죠.

응용 그대로 출력하면 마이크로초까지 나와 지저분합니다. f-string의 **서식 지정자**로 원하는 형태만 뽑아낼 수 있습니다.

```
import datetime

now = datetime.datetime.now()
print(f"{now:%Y-%m-%d}")          # 날짜만
print(f"{now:%H:%M:%S}")          # 시각만
print(f"{now:%Y-%m-%d %H:%M:%S}") # 둘 다
print(f"점검 시각: {now:%Y년 %m월 %d일 %H시 %M분}")
```

```
▶ 2026-08-05
▶ 11:19:45
▶ 2026-08-05 11:19:45
▶ 점검 시각: 2026년 08월 05일 11시 19분
```

자주 하는 실수

`datetime.datetime.now()`에서 **datetime**이 두 번 나오는 게 혼란 지점입니다. 앞은 **모듈**, 뒤는 그 안의 **클래스**입니다. `from datetime import datetime`으로 가져오면 `datetime.now()`로 짧아지지만, 그러면 모듈인지 클래스인지 더 헷갈립니다.

모듈 문서 읽는 법 — 다 외울 필요 없다

교안이 강조한 부분입니다 — "**다 외울 필요 없음. 필요할 때 찾는 법만 알면 됨.**" 파이썬에는 도구 두 개가 내장되어 있습니다.

```
dir과 help
import math

print(dir(math))          # 이 모듈에 어떤 기능이 있나 — 목록
help(math.sqrt)           # 이 기능은 어떻게 쓰나 — 설명서
```

```
▶ ['acos', 'ceil', 'cos', 'e', 'floor', 'pi', 'sqrt', ...]
▶ sqrt(x, /)
▶     Return the square root of x.
```

개념

`help()`가 보여주는 그 설명문이 바로 **어제 배운 docstring**입니다. 파이썬 표준 라이브러리를 만든 사람들도 함수마다 docstring을 달아 두었고, `help()`가 그걸 꺼내 보여주는 것이죠. **어제 배운 습관이 왜 중요한지**가 여기서 드러납니다 — 내가 만든 함수도 docstring을 달아두면 `help(내함수)`가 똑같이 동작합니다.

표준 라이브러리 vs 외부 라이브러리

구분	표준 라이브러리	외부 라이브러리
예시	<code>math</code> · <code>os</code> · <code>csv</code> · <code>random</code> · <code>datetime</code>	<code>NumPy</code> · <code>Pandas</code> · <code>Matplotlib</code>
설치	불필요 — 파이썬과 함께 설치됨	<code>pip install</code> 로 따로 설치
사용	<code>import</code> 로 바로 사용	설치 후 <code>import</code>

주의

`ModuleNotFoundError: No module named pandas` 같은 오류는 대개 외부 라이브러리를 설치하지 않아서 납니다. 오타가 아니라 설치 문제죠. 터미널에서 `pip install pandas`를 하면 해결됩니다. 반대로 `No module named maths`처럼 표준 라이브러리 이름에서 오류가 나면 그건 오타입니다 — `math`인데 `maths`로 쓴 경우죠.

STEP 4

경로 — 절대·상대·작업 디렉토리

파일을 못 찾는 진짜 이유

폴더는 **나무를 거꾸로 세운 계층 구조**이고, 경로는 그 가치를 따라가는 길을 적은 것입니다. 슬래시는 "**폴더 안으로 들어간다**"는 의미입니다.

경로 = 가치를 따라간 길

documents/project/data/08_press.csv

```
documents/  
└─ project/  
   └─ data/  
      └─ 08_press.csv
```

주의

윈도우는 역슬래시(\), 맥·리눅스는 슬래시(/)를 씁니다. 이 차이 때문에 윈도우에서 짠 코드가 맥에서 안 돌기도 하는데, STEP 5에서 배울 `os.path.join`이 자동으로 맞춰 줍니다. 그래서 경로를 문자열로 직접 이어 붙이지 말고 `join`을 쓰라는 것입니다.

절대경로와 상대경로

구분	절대경로	상대경로
기준	최상위 폴더부터 끝까지 적음	현재 위치에서 출발
예시	C:\Users\me\project\data\a.csv	data/a.csv
길이	길다 — 우편 주소처럼	짧다 — "여기서 오른쪽으로" 식
폴더 이동 시	옳기면 오류	옳게도 동작

개념

실무는 **상대경로**를 훨씬 자주 씁니다. 내 컴퓨터에서 C:\Users\Minho\...로 짠 코드를 동료에게 주면 그 사람 컴퓨터에는 그 폴더가 없어 바로 터지니까요. 상대경로로 짜두면 **프로젝트 폴더째로 옮겨도 그대로 동작**합니다. 참고로 **점 하나(.)**는 현재 폴더, **점 두 개(..)**는 상위 폴더를 뜻합니다 — `../data/a.csv`는 "한 단계 위로 올라가서 data 폴더의 a.csv"입니다.

작업 디렉토리 — 오늘 가장 중요한 개념

작업 디렉토리(cwd, current working directory)는 코드가 "지금 여기 서 있다"고 여기는 폴더입니다. 상대경로는 모두 이 작업 디렉토리에서 출발합니다.

자주 하는 실수

교안이 굵게 표시한 문장입니다 — **"작업 디렉토리는 코드 파일 위치와 항상 같지는 않습니다."** 그래서 파일 바로 옆에 데이터를 뒀는데도 못 찾는 일이 생깁니다. 같은 코드가 어제는 됐는데 오늘은 `FileNotFoundError`가 나는 이유도 대개 이것입니다 — **어디서 실행했느냐**가 달라진 거죠.

TERMINAL — 같은 코드, 다른 결과

같은 파일을 두 위치에서 실행하면

```
$ cd ~/project          # 프로젝트 루트에서
$ python Python/18_module_2.py
/home/me/project        ← 작업 디렉토리
a.ipynb                  ← Ipython 폴더를 찾음 ✓
```

```
$ cd ~/project/Python   # Python 폴더 안에서
$ python 18_module_2.py
FileNotFoundError: No such file or directory: 'Ipython'    x
```

코드는 한 글자도 안 바뀌었는데 결과가 달라집니다. "Ipython"라는 상대경로가 작업 디렉토리 기준이기 때문입니다.

그래서 파일을 읽기 전에 **현재 위치부터 확인하는 습관**이 중요합니다 — 그 도구가 STEP 5의 `os.getcwd()`입니다.

STEP 5

os 모듈로 경로 다루기

getcwd · listdir · join · exists

메서드 `os.getcwd()` 현재 작업 폴더 확인

정의 지금 코드가 기준으로 삼는 폴더의 **절대경로**를 문자열로 돌려줍니다. get current working directory의 줄임말입니다.

형태 `os.getcwd()`

```
import os

cwd = os.getcwd()
print(cwd)
print(type(cwd).__name__)

▶ /home/me/project
▶ str
```

왜 쓰나 파일을 못 찾을 때 가장 먼저 찍어 볼 것입니다. 상대경로가 어디서 출발하는지 알려주므로, FileNotFoundError의 원인을 즉시 좁힐 수 있습니다.

응용 실무에서는 파일을 읽기 전에 현재 위치와 파일 존재를 함께 확인하는 세 줄을 습관처럼 넣습니다. 오류 메시지를 스스로 만들어 두면 나중에 원인 찾기가 훨씬 쉬워집니다.

```
import os

path = "data/08_press.csv"
print("현재 위치:", os.getcwd())
print("찾는 경로:", os.path.abspath(path))    # 상대경로가 실제로 어디를 가리키는지
print("존재 여부:", os.path.exists(path))

▶ 현재 위치: /home/me/project
▶ 찾는 경로: /home/me/project/data/08_press.csv
▶ 존재 여부: True
```

개념

`os.path.abspath()`는 상대경로를 절대경로로 바꿔 보여줍니다. "내가 쓴 상대경로가 실제로 어디를 가리키는가"를 확인하는 데 `getcwd`보다 직접적입니다.

메서드 `os.listdir()` 폴더 안 목록 보기

정의 폴더 안의 **모든 항목 이름**을 리스트로 돌려줍니다. 파일과 폴더를 **구분하지 않고 전부** 담습니다.

형태 `os.listdir(경로)` · `os.listdir()` ← 생략하면 현재 폴더

```
import os

items = os.listdir()          # 현재 폴더
print(items)

items = os.listdir("Ipynb")   # 특정 폴더
for name in items:
    print(name)
```

```
▶ ['data', '08_01_example.py', 'Ipynb', 'Remind']
▶ a.ipynb
```

왜 쓰나 데이터 폴더에 어떤 파일이 있는지 **코드로 훑어볼 때** 씁니다. 파일이 수백 개면 손으로 확인할 수 없으니까요.

응용 **폴더도 함께 나온다는 점**이 함정입니다. "파일 개수"를 세려면 `os.path.isfile()`로 걸러야 정확합니다. 실습 5에서 실제로 이 문제가 나오는데, STEP 8에서 다룹니다.

```
import os

items = os.listdir()
files = [n for n in items if os.path.isfile(n)]
folders = [n for n in items if os.path.isdir(n)]

print("전체 항목:", len(items))
print("파일 :", len(files), files)
print("폴더 :", len(folders), folders)
```

```
▶ 전체 항목: 4
▶ 파일 : 1 ['08_01_example.py']
▶ 폴더 : 3 ['data', 'Ipynb', 'Remind']
```

자주 하는 실수

`listdir`이 돌려주는 것은 **이름만**이지 경로가 아닙니다. "a.csv"이지 "data/a.csv"가 아니죠. 그래서 그 파일을 실제로 열려면 `os.path.join`으로 폴더 이름을 앞에 붙여야 합니다.

메서드 `os.path.join()` 경로를 안전하게 잇기

정의 폴더 이름과 파일 이름을 **운영체제에 맞는 구분자**로 이어 붙입니다.

형태 `os.path.join("폴더", "하위폴더", "파일명")`

```
import os

path = os.path.join("data", "08_press.csv")
print(path)

path = os.path.join("Remind", "함수.pdf")
print(path)

print(os.path.join("a", "b", "c", "d.csv"))    # 여러 단계도 가능
```

- ▶ data/08_press.csv
- ▶ Remind/함수.pdf
- ▶ a/b/c/d.csv

왜 쓰나 윈도우와 맥·리눅스의 구분자 차이를 자동으로 맞춰 주기 때문입니다. 윈도우에서 실행하면 `data\08_press.csv`, 맥에서 실행하면 `data/08_press.csv`가 나옵니다. 같은 코드가 어디서든 동작하죠.

응용 문자열 덧셈으로 직접 이어 붙이면 구분자를 빠뜨리거나 두 번 넣는 실수가 생깁니다. `join`은 그런 실수를 원천 차단합니다.

```
import os

folder = "data"
name = "08_press.csv"

# ✕ 직접 이어 붙이기 — 구분자를 빠뜨리기 쉽다
print(folder + name)
print(folder + "/" + name)    # 동작은 하지만 OS에 종속

# ✓ join — 안전하고 이식성 있음
print(os.path.join(folder, name))
```

- ▶ data08_press.csv ← 구분자 누락!
- ▶ data/08_press.csv
- ▶ data/08_press.csv

주의

`listdir` + `join` 조합이 정석입니다 — **목록을 이름으로 받아, join으로 경로를 만들어, 그 경로로 파일을 연다.**
오늘 실습 6이 정확히 이 흐름입니다.

메서드 `os.path.exists()` 파일이 있는지 확인

정의 그 경로에 파일이나 폴더가 있으면 `True`, 없으면 `False`를 돌려줍니다.

형태 `os.path.exists(경로)`

```
import os

print(os.path.exists("data/08_press.csv"))
print(os.path.exists("data/없는파일.csv"))
print(type(os.path.exists(".")).__name__)
```

- ▶ `True`
- ▶ `False`
- ▶ `bool`

왜 쓰나 교안 표현으로 "읽기 전에 확인하면 프로그램이 멈추지 않는다"입니다. 없는 파일을 바로 열면 `FileNotFoundError`로 그 자리에서 끝나지만, 확인하고 열면 없을 때 다른 처리를 할 수 있습니다.

응용 어제 배운 조건문과 묶어 쓰는 것이 기본형입니다. 그리고 **파일인지 폴더인지 구분**하려면 `isfile·isdir`을 씁니다 — `exists`는 둘 다 `True`를 돌려주니까요.

```
import os

path = os.path.join("data", "08_press.csv")

if os.path.exists(path):
    print(f"파일 있음 — {path}")
else:
    print(f"파일 없음 — {path}")

print("파일인가:", os.path.isfile("data"))    # 폴더라서 False
print("폴더인가:", os.path.isdir("data"))    # True
```

- ▶ 파일 있음 — `data/08_press.csv`
- ▶ 파일인가: `False`
- ▶ 폴더인가: `True`

자주 하는 실수

`exists()`는 이미 `bool`을 돌려줍니다. `bool(os.path.exists(path))`처럼 한 번 더 감쌀 필요가 없습니다 — 실습 4에서 실제로 나온 부분이라 STEP 8에서 다시 다룹니다.

os 4종 한 줄 정리

기능	돌려주는 것	언제
<code>os.getcwd()</code>	현재 작업 폴더 (문자열)	파일을 못 찾을 때 가장 먼저
<code>os.listdir(경로)</code>	폴더 안 항목 이름 (리스트)	어떤 파일이 있나 훑을 때
<code>os.path.join(a, b)</code>	이어 붙인 경로 (문자열)	경로를 만들 때 — 항상
<code>os.path.exists(경로)</code>	True / False	열기 전에 확인할 때

STEP 6

파일 입출력 시작

open · 모드 · 인코딩 · with

여기부터가 교안 08_02입니다. 오늘은 도입부까지만 나왔습니다 — 파일을 여는 법과 인코딩 개념까지죠. 실제로 읽고 쓰는 것(read·csv.reader·csv.writer)은 내일입니다.

개념

교안이 정리한 한 문장이 앞으로의 방향입니다 — "앞서 import·os로 파일을 찾았다면, 이번 시간에는 그 파일을 열고·읽고·처리하고·저장한다. 둘을 합치면 데이터 처리의 완전한 한 사이클이 완성된다." 오늘은 그 사이클의 앞쪽 절반, 즉 찾기와 열기까지입니다.

왜 데이터를 파일로 주고받나

설비 데이터는 하루에도 수만 줄입니다. 코드에 직접 적을 수 없죠. 그래서 데이터는 파일에, 코드는 따로 둡니다.

장점	뜻	실제로는
코드 고정	데이터가 바뀌어도 코드는 그대로	오늘 파일이든 어제 파일이든 같은 코드로 분석
공유	결과를 파일로 저장해 전달·재사용	다음 작업에 이어 쓰기 좋다

내장함수 open() 파일을 여는 연결 통로

정의 파일명·모드·인코딩을 주면 **파일 객체**(파일을 다룰 연결 통로)를 돌려줍니다. **내장 함수라 import가 필요 없습니다.**

형태 open(파일명, 모드, encoding="utf-8")

```
f = open("data/08_press.csv", "r", encoding="utf-8")

print(type(f).__name__)

f.close()          # 열었으면 닫는다

▶ TextIOWrapper
```

왜 쓰나 open이 돌려주는 것은 **파일 자체가 아니라 연결 통로**입니다. 수도꼭지를 튼 것에 가깝죠 — 물(내용)은 아직 안 나왔고, 나오게 할 준비만 된 상태입니다. 실제로 내용을 꺼내는 것은 read·readline 등이고 내일 배웁니다.

응용 `open`은 `print·len`처럼 **내장 함수**라 `import` 없이 바로 쓴다는 점이 중요합니다. `os`나 `csv`는 `import`가 필요한데 `open`은 아니죠. STEP 3에서 말한 "얼마나 기본적인 기능인가"로 갈리는 그 경계입니다.

```
import os

path = os.path.join("data", "08_press.csv")

if os.path.exists(path):
    # ① 있는지 확인 (STEP 5)
    f = open(path, "r", encoding="utf-8") # ② 열기
    print("열기 성공:", type(f).__name__)
    f.close() # ③ 닫기
else:
    print("파일이 없습니다:", path)
```

▶ 열기 성공: `TextIOWrapper`

자주 하는 실수

없는 파일을 "r" 모드로 열면 `FileNotFoundError`로 즉시 멈춥니다. STEP 5의 `exists`로 먼저 확인하는 이유가 이것입니다.

문법 파일 모드 `r` · `w` · `a` 무슨 목적으로 열 것인가

정의 `r`은 읽기, `w`는 새로 쓰기(**기존 내용 삭제**), `a`는 이어 쓰기입니다.

왜 쓰나 모드를 잘못 고르면 **데이터가 날아갑니다**. 교안이 경고 표시까지 붙인 이유가 있습니다.

응용 `w`와 `a`의 차이를 반드시 몸으로 기억해야 합니다. 로그를 계속 쌓아야 하는데 `w`로 열면 **매번 앞의 기록이 지워집니다**. 점검 이력처럼 누적되어야 하는 데이터는 반드시 `a`입니다.

```
# x w 모드 — 실행할 때마다 앞 기록이 사라진다
with open("log.txt", "w", encoding="utf-8") as f:
    f.write("점검 1회\n")

# ✓ a 모드 — 뒤에 이어 붙는다
with open("log.txt", "a", encoding="utf-8") as f:
    f.write("점검 2회\n")
```

주의

모드를 생략하면 기본값이 `"r"`입니다. 즉 `open(path)`는 `open(path, "r")`과 같습니다 — 어제 배운 **기본값 인자**가 여기서도 쓰이고 있는 것입니다.

모드	의미	동작	쓸 때
r	읽기 (기본값)	내용 읽기 — 없는 파일이면 오류	데이터를 불러올 때
w	새로 쓰기	기존 내용 삭제 후 작성	결과 파일을 새로 만들 때
a	이어 쓰기	기존 뒤에 추가	로그를 누적할 때

문법 인코딩 (utf-8) 글자를 숫자로 바꾸는 약속

정의 컴퓨터는 글자를 숫자로 바꿔 저장합니다. 그 약속이 인코딩이고, 저장할 때와 읽을 때의 약속이 다르면 **글자가 깨집니다**.

```
# 오늘날의 표준
f = open("data.csv", "r", encoding="utf-8")

# 윈도우 엑셀로 저장한 CSV는 cp949일 수 있다
f = open("data.csv", "r", encoding="cp949")
```

왜 쓰나 강사님이 시간을 들여 설명하신 부분입니다 — "한글이 깨져 나온다면 그건 utf-8이 아니라는 뜻"입니다.

원인을 모르면 몇 시간을 헤매지만, 알면 인코딩 한 글자만 바꾸면 끝납니다.

응용 한글 인코딩에는 역사가 있습니다. 강사님이 EUC-KR·CP949 이야기를 하셨는데, 메모리가 귀하던 시절 한글을 더 적은 바이트로 저장하려고 만든 방식들입니다. 지금도 오래된 웹사이트나 윈도우 엑셀에서 저장한 CSV에 남아 있습니다.

```
# 한글이 깨지면 순서대로 시도해 본다
for enc in ["utf-8", "cp949", "euc-kr"]:
    try:
        with open("data.csv", "r", encoding=enc) as f:
            first = f.readline()
            print(f"{enc} → {first[:20]}")
            break
    except UnicodeDecodeError:
        print(f"{enc} → 실패")
```

개념

utf-8이 오늘날의 표준입니다. 거의 모든 환경에서 안전하므로 **기본은 utf-8로 두고, 깨지면 cp949로 바꿔 보는** 순서가 좋습니다. 참고로 영문·숫자만 있으면 어느 인코딩이든 대체로 잘 읽힙니다 — 앞부분(아스키 영역)이 겹치기 때문입니다. 문제는 **항상 한글에서 터집니다**.

키워드 with open() as f 자동으로 닫아 주는 형태

정의 with 블록이 끝나면 **파일이 자동으로 닫힙니다**. close()를 쓸 필요가 없고, 오류가 나도 안전하게 닫힙니다.

형태 `with open(경로, 모드, encoding="utf-8") as f:` (블록 안에서 파일을 다룬다)

```
with open("data/08_press.csv", "r", encoding="utf-8") as f:
    for line in f:
        print(line.strip())
```

블록 밖으로 나가면 자동으로 닫힘 — `close()` 불필요

```
▶ 설비ID, 시각, 진동X, 진동Y, 전류, 상태
▶ PRESS-01, 2022-07-12 00:00:00.019, 0.117, -0.1764, 192.3387, 0
▶ ...
```

왜 쓰나 파일을 닫아야 하는 이유가 두 가지입니다 — ① **자원 반납**: 안 닫으면 쌓여서 나중에 파일을 못 열 수 있고, ②

저장 보장: 쓰기에서 닫지 않으면 임시 공간의 내용이 실제 파일에 기록되지 못하고 사라집니다.

응용 `close()`를 직접 쓸 수도 있지만 **깜빡하기 쉽습니다**. 그리고 중간에 오류가 나면 `close()` 줄까지 도달하지 못해 파일이 열린 채 남습니다. `with`는 오류가 나도 블록을 벗어나는 순간 닫아 주므로 **실무의 기본 형태**입니다.

```
# x 중간에 오류가 나면 close()에 도달 못 함
f = open("data.csv", "r", encoding="utf-8")
value = int("abc")          # ValueError! 여기서 중단
f.close()                   # 실행되지 않는다

# ✓ with — 오류가 나도 자동으로 닫힌다
with open("data.csv", "r", encoding="utf-8") as f:
    value = int("abc")      # 오류가 나도

# 블록을 벗어나며 파일은 닫힌다
```

주의

강사님 표현으로 "**열기도 했으니까 풀어 주는 닫힘**"입니다. `with`가 그 닫힘을 자동으로 해 주므로 **앞으로는 거의 항상 `with open`을 쓰게 됩니다**. `f = open(...)` 형태는 원리를 이해하기 위한 것이고, 실전 코드는 `with`입니다.

파일 입출력 전체 흐름 — 오늘은 여기까지

파일 열기
`with open`



모드 선택
`r · w · a`



다루기
`reader / writer`



자동 닫힘
블록 종료

주의

세 번째·네 번째 칸을 회색으로 둔 이유는 **오늘 진도가 거기까지 가지 않았기** 때문입니다. 강사님이 수업 끝에 이렇게 예고하셨습니다 — **"내일 본격적으로 현재 있는 코드를 기반으로 읽고 쓰는 작업을 배우고, 이어서 에러 핸들링까지. 왜냐하면 파일을 다루다 보면 에러가 먼저 발생할 겁니다."** `read·readline·readlines`, `csv.reader·csv.writer`, 그리고 예외 처리가 내일 몫입니다.

STEP 7

내 코드 점검 — 수업 코드 2개

작업 디렉토리에 걸린 코드

오늘 만든 수업 파일은 `18_module_1 copy.py`와 `18_module_2.py` 두 개입니다. 내용은 정확하고, 주석도 잘 달아 두셨습니다. 짚을 곳은 **파일 이름 하나와 경로 하나**입니다.

18_module_1 copy.py — import 세 방식이 정확히 기록됨

세 방식이 순서대로 잘 정리되어 있고, 주석도 각 방식의 요점을 정확히 짚었습니다. `datetime`까지 붙여 놓은 것도 좋습니다.

제출 코드 — 정확함

```
import math
result = math.sqrt(16)          # 모듈이름.함수() 식으로 호출

from math import sqrt
result = sqrt(16)              # 이젠 sqrt만 불러도 됩니다

import math as mt
result = mt.sqrt(16)          # 별칭으로 가져온 모듈 이름

import datetime as dt
now = dt.datetime.now()
print(now)                    # 2026-08-05 11:19:45.776780
print(type(now))              # <class 'datetime.datetime'>
```

```
▶ 4.0
▶ 4.0
▶ 4.0
▶ 2026-08-05 11:19:45.776780
▶ <class 'datetime.datetime'>
```

주의

파일 이름이 `18_module_1 copy.py`입니다. 파일 탐색기에서 복사했을 때 붙는 이름인데, 두 가지 문제가 있습니다 — ① 이름에 공백이 들어가 터미널에서 실행할 때 따옴표가 필요하고(`python "18_module_1 copy.py"`), ② 나중에 이 파일을 모듈로 **import**할 수 없습니다. 모듈 이름에는 공백을 쓸 수 없기 때문입니다 (어제 배운 함수 이름 규칙과 같습니다). `18_module_1.py`로 바꿔 두시는 게 좋겠습니다.

18_module_2.py — 작업 디렉토리에 의존하는 코드

os의 네 기능을 정확히 썼습니다. 다만 `os.listdir("Ipynb")` 줄이 어디서 실행하느냐에 따라 성공하기도, 실패하기도 합니다.

제출 코드 — 18_module_2.py

```
import os

current_working_directory = os.getcwd()
print(current_working_directory)

file_list = os.listdir("Ipynb")      # ← 상대경로: 작업 디렉토리 기준
for file_name in file_list:
    print(file_name)

path = os.path.join("Python", "08_press.csv")
if os.path.exists(path):
    print("파일 있음", path)
```

실행 위치	<code>os.getcwd()</code>	<code>os.listdir("Ipynb")</code>
프로젝트 루트에서	<code>~/project</code>	성공 — Ipynb 폴더를 찾음
Python 폴더 안에서	<code>~/project/Python</code>	<code>FileNotFoundError</code>

개념

STEP 4에서 배운 "작업 디렉토리는 코드 파일 위치와 항상 같지는 않다"가 그대로 재현됩니다. 코드 파일은 `Python/` 안에 있지만, 상대경로 `"Ipynb"`는 코드 파일 위치가 아니라 작업 디렉토리에서 출발하기 때문입니다. VS Code에서 실행 버튼을 누르면 대개 프로젝트 루트가 작업 디렉토리가 되어 성공하지만, 터미널에서 `cd Python` 후 실행하면 실패합니다. 같은 코드가 어제는 됐는데 오늘은 안 되는 전형적인 상황입니다.

고친 코드 — 위치를 확인하고 안내

```
import os

# 방법 ① — 실행 전에 위치를 확인하고 안내한다
target = "Ipynb"
if not os.path.exists(target):
    print(f"[안내] 현재 위치는 {os.getcwd()} 입니다.")
    print(f"여기에 {target} 폴더가 없습니다. 프로젝트 루트에서 실행하세요.")
else:
    for name in os.listdir(target):
        print(name)
```

- ▶ [안내] 현재 위치는 /home/me/project/Python 입니다.
- ▶ 여기에 Ipynb 폴더가 없습니다. 프로젝트 루트에서 실행하세요.

주석에 # 만약 C:\Users\Nedpark\바탕화면\sample 이라고 절대경로 예시를 적어 두셨는데, 이 방향도 하나의 해결책입니다. 다만 **절대경로는 옮기면 깨진다**는 STEP 4의 단점이 있어, 실무에서는 위처럼 **상대경로 + 존재 확인**을 더 자주 씁니다.

STEP 8

실습 점검 — 기본 4 · 선택 2

버그 4건과 재귀 대신 요구사항대로

오늘 실습과제는 기본문제 4개(실습 1·2·4·5)와 선택문제 2개(실습 3·6)입니다. 여섯 개를 모두 제출하셨습니다 — 요건을 충분히 넘겼습니다. 다만 몇 군데가 요구사항과 어긋나 있어 짚습니다.

구분	문제	제출	상태
기본	실습 1 · import 세 방식	○	정확
기본	실습 2 · 표준 라이브러리로 센서값	○	원본 소실
기본	실습 4 · os로 파일 존재 확인	○	④ 미이행 · bool 중복
기본	실습 5 · datetime으로 점검 기록	○	"파일 수"가 부정확
선택	실습 3 · os로 폴더 목록 살펴보기	○	리스트 이중 포장 버그
선택	실습 6 · csv 파일만 골라내기	○	전역 변수 · 재귀

실습 3 — 리스트를 리스트로 한 번 더 감쌌습니다

가장 눈에 띄는 버그입니다. `os.listdir()`이 이미 리스트를 돌려주는데, 대괄호로 한 번 더 감쌌습니다.

제출 코드 — 실습 3

```
ls = [os.listdir()]          # ← 대괄호가 하나 더
print("폴더 목록 리스트:", ls)
```

▶ 폴더 목록 리스트: [['data', '08_01_example.py', 'Ipynb', 'Remind']]

	제출 코드	올바른 코드
결과	<code>[['a.csv', 'b.txt', 'sub']]</code>	<code>['a.csv', 'b.txt', 'sub']</code>
<code>len()</code>	항상 1 — 바깥 리스트의 길이	실제 항목 수
for로 순회하면	리스트 하나가 통째로 나온다	이름이 하나씩 나온다

자주 하는 실수

출력만 보면 대괄호가 하나 더 있을 뿐이라 넘어가기 쉽습니다. 하지만 `len(ls)`가 항상 1이고 `for x in ls:`가 한 번만 돌립니다. 실습 5에서 개수를 세거나 실습 6에서 순회하려고 이 변수를 썼다면 그대로 사고로 이어졌을 겁니다. 어제 배운 "여러 입력으로 테스트했는가" 체크리스트에 걸리는 항목이죠.

실습 5 — "파일 수"가 아니라 "항목 수"입니다

요구사항 ②는 "`listdir`로 폴더 파일 수를 구하기"이고 예상 결과는 파일 3개, 점검 시각 ...입니다. 그런데 `os.listdir()`은 폴더도 함께 돌려줍니다.

제출 코드 — 실습 5

```
num_dir = len(os.listdir())          # 폴더까지 포함된 개수
now = datetime.datetime.now()
print(f"파일 수: {num_dir} | 현재 시각: {now}")
```

▶ 파일 수: 4 | 현재 시각: 2026-08-05 23:45:45.622343

▶

▶ ↑ 실제 파일은 1개, 나머지 3개는 폴더

고친 코드 — 예상 결과 형식과 일치

```
import os, datetime
```

```
items = os.listdir()
files = [n for n in items if os.path.isfile(n)]      # 파일만 걸러낸다

now = datetime.datetime.now()
print(f"파일 {len(files)}개, 점검 시각 {now:%Y-%m-%d %H:%M:%S}")
```

▶ 파일 1개, 점검 시각 2026-08-05 23:46:05

개념

두 가지를 함께 고쳤습니다. ① `os.path.isfile()`로 **파일만** 세고, ② 시각을 f-string 서식으로 **마이크로초를 잘라** 냈습니다. 예상 결과가 점검 시각 2026-... 형식이었으니 초 단위까지가 적당합니다. 변수 이름도 `num_dir`(디렉토리 수)보다 `files`가 정확합니다 — **이름이 내용과 맞아야 한다**는 어제 원칙입니다.

실습 4 — ④단계가 빠지고 bool이 중복입니다

요구사항 ④는 "if로 있으면·없으면 다른 메시지 출력"이었는데, True/False만 출력하셨습니다. 그리고 `os.path.exists()`는 이미 bool을 돌려주므로 `bool()`로 감쌀 필요가 없습니다.

제출 코드 — 실습 4

```
def find_path(path):
    for p in path:
        print(f"{p} | 경로 참, 거짓 확인: {bool(os.path.exists(p))}")
# ↑ bool() 중복 · if 분기 없음
```

- ▶ Remind/함수.pdf | 경로 참, 거짓 확인: True
- ▶ Remind/함성.pdf | 경로 참, 거짓 확인: False

고친 코드 — ④ 반영

```
import os

def check_paths(paths):
    """경로 목록을 받아 각각의 존재 여부를 안내한다."""
    for p in paths:
        if os.path.exists(p):
            print(f"파일 있음 — {p}")
        else:
            print(f"파일 없음 — {p}")

targets = (
    os.path.join("Remind", "함수.pdf"),
    os.path.join("Remind", "함성.pdf"),
)
check_paths(targets)
```

- ▶ 파일 있음 — Remind/함수.pdf
- ▶ 파일 없음 — Remind/함성.pdf

응용

매개변수 이름을 `path`에서 `paths`(복수형)로 바꾼 것도 의미가 있습니다. 넘어오는 것이 **경로 하나가 아니라 여러 개**이므로 복수형이 정확합니다. 어제 배운 "매개변수 이름은 무엇이 들어올지 드러나게"에 해당합니다. 그리고 docstring을 달아 두면 나중에 `help(check_paths)`로 바로 확인할 수 있습니다.

실습 2 — 원본 값이 사라집니다

요구사항 ③은 "**math 모듈로 그 값을 가공(제곱근)**"인데, 원본 리스트를 그 자리에서 덮어썼습니다. 그래서 ④단계 "다시 실행하면 값이 달라지는지 확인"을 할 때 **비교할 원본이 없습니다**.

제출 코드 — 실습 2

```
for idx, val in enumerate(sens_val):
    sens_val[idx] = round(math.sqrt(val), 4)    # ← 원본 자리에 덮어쓰기

print(f"센서값 리스트 재출력 결과: {sens_val}")
```

- ▶ sens_val 랜덤 20개 값: [34, 19, 50, ...]
- ▶ 센서값 리스트 재출력 결과: [5.831, 4.3589, 7.0711, ...]
- ▶
- ▶ ↑ 원본 34, 19, 50 은 어디에도 남지 않았다

고친 코드 — 원본 보존

```
import random
import math

raw = [random.randint(1, 100) for _ in range(20)]    # 원본은 그대로 둔다
roots = [round(math.sqrt(v), 4) for v in raw]        # 가공값은 새 리스트로

print("원본 :", raw[:5], "...")
print("제곱근:", roots[:5], "...")

# ④ 다시 실행하면 값이 달라지는지 — 원본이 남아 있어야 비교가 된다
print("원본 첫 값:", raw[0], "→ 제곱근:", roots[0])
```

- ▶ 원본 : [34, 19, 50, 11, 82] ...
- ▶ 제곱근: [5.831, 4.3589, 7.0711, 3.3166, 9.0554] ...
- ▶ 원본 첫 값: 34 → 제곱근: 5.831

개념

9일차에 배운 `sort()` vs `sorted()`와 같은 구도입니다 — 원본을 직접 바꿀 것인가, 새로 만들 것인가. **데이터 분석에서는 대개 원본을 남기는 쪽이 안전합니다. 가공 결과가 이상할 때 원본과 대조해 봐야 원인을 찾을 수 있으니까요. 실제 설비 데이터라면 원본을 덮어쓰는 순간 되돌릴 방법이 없습니다.**

실습 3 · 6 — 재귀 대신 요구사항대로

두 문제 모두 **재귀 함수**로 푸셨습니다. 하위 폴더까지 전부 뒤져 CSV를 찾는 코드죠. 아직 배우지 않은 기법을 찾아 쓴 것은 좋습니다. 다만 요구사항은 훨씬 단순합니다 — `listdir` 한 번 + `for-if` 한 번입니다.

	제출 코드	요구사항
방식	재귀 — 하위 폴더까지 전부 탐색	<code>listdir</code> 한 번 — 한 폴더만
실습 3 ④	csv만 출력 (목록 하나씩 출력이 빠짐)	목록을 하나씩 출력하고 csv만 골라 출력
실습 6 결과	전역 변수 <code>dir_list</code> 에 누적	함수 안 리스트에 모아 return

자주 하는 실수

실습 6의 `dir_list`가 **함수 밖에 있는 전역 변수**인데 함수 안에서 `append`로 채웁니다. 어제 STEP 9에서 배운 **"함수는 깔끔한 통로로만 소통한다"**에 정면으로 어긋나는 구조죠. 이 함수는 혼자서는 동작하지 않고, 두 번 호출하면 앞의 결과가 그대로 남아 **결과가 누적되어 틀립니다**. 재귀 자체는 문제가 아니지만, **결과를 전역 변수로 주고받는 것은 고쳐야 합니다**.

실습 3 · 6 — 요구사항대로 (전역 변수 없이)

```
import os

def find_csv(folder):
    """폴더 안의 csv 파일 경로를 리스트로 돌려준다."""
    result = []  # 함수 안에서 만들고
    for name in os.listdir(folder):
        print("-", name)  # ④ 목록을 하나씩 출력
        if name.endswith(".csv"):
            result.append(os.path.join(folder, name))
    return result  # return으로 내보낸다
```

```
print("현재 위치:", os.getcwd())
csv_files = find_csv("data")
print("CSV만:", csv_files)
```

```
▶ 현재 위치: /home/me/project
▶ - 08_press.csv
▶ CSV만: ['data/08_press.csv']
```

응용

하위 폴더까지 뒤지고 싶다면 재귀를 쓰되 **결과는 return으로 주고받으면** 됩니다.

`result.extend(find_csv(full_path))` 한 줄이면 자기 자신이 돌려준 목록을 이어 붙일 수 있죠. 참고로 파이썬에는 이 일을 대신 해 주는 `os.walk()`와 `glob` 모듈이 이미 있습니다 — STEP 1에서 말한 **"직접 만들기보다 가져다 쓰기"**가 여기에도 적용됩니다. 지금 단계에서는 요구사항대로 단순하게 푸는 편이 낫고, 나중에 필요해지면 그때 찾아 쓰면 됩니다.

파일 정리 — import 위치와 실습 순서

항목	현재	권장
import 위치	실습마다 흩어져 있음 (os를 3번 import)	파일 맨 위에 한 번만 모아 적기
import random, math	한 줄에 두 개	줄을 나눠 각각 한 줄씩 (PEP 8)
실습 순서	1 → 2 → 4 → 5 → 3 → 6	번호 순서대로 배치

주의

같은 모듈을 여러 번 import해도 오류는 안 나고 성능 손해도 없습니다 — 파이썬이 이미 가져온 모듈은 다시 읽지 않으니깐요. 그래도 맨 위에 모으는 이유는 읽는 사람이 이 파일이 어떤 도구를 쓰는지 한눈에 보기 위해서입니다. 그리고 어제 강사님이 말씀하신 대로 실습은 **문제마다 파일을 나누는** 것이 더 좋습니다 — 그러면 import 정리도 자연스럽게 해결됩니다.

치트시트

모듈 · 경로 · 파일 한 장 요약

import 세 방식

	문법	사용	언제
통째로	<code>import math</code>	<code>math.sqrt(16)</code>	출처를 명확히 하고 싶을 때 (기본)
일부만	<code>from math import sqrt</code>	<code>sqrt(16)</code>	한두 기능을 자주 쓸 때
별칭	<code>import numpy as np</code>	<code>np.array(...)</code>	모듈명이 길 때 · 관례가 있을 때
별표	<code>from math import *</code>	전부 한꺼번에	실무 회피 — 이름 충돌 · 출처 가림

표준 라이브러리 5종

모듈	대표 기능	오늘 쓴 예
<code>math</code>	<code>sqrt</code> · <code>ceil</code> · <code>floor</code> · <code>pi</code>	진동 X·Y를 합친 크기 계산
<code>random</code>	<code>randint</code> · <code>choice</code> · <code>random</code> · <code>seed</code>	가짜 센서값 20개 만들기
<code>datetime</code>	<code>datetime.now()</code>	점검 시각 기록
<code>os</code>	<code>getcwd</code> · <code>listdir</code> · <code>path.join</code> · <code>path.exists</code>	데이터 폴더 탐색
<code>csv</code>	<code>reader</code> · <code>writer</code>	내일

경로 · os

	문법	돌려주는 것	비고
메서드	<code>os.getcwd()</code>	현재 작업 폴더 (문자열)	못 찾을 때 가장 먼저
메서드	<code>os.listdir(경로)</code>	항목 이름 (리스트)	폴더도 함께 나온다
메서드	<code>os.path.join(a, b)</code>	이어 붙인 경로 (문자열)	OS 구분자 자동 처리
메서드	<code>os.path.exists(경로)</code>	True / False	이미 bool — 감쌀 필요 없음
메서드	<code>os.path.isfile</code> / <code>isdir</code>	True / False	파일 · 폴더 구분할 때

	문법	돌려주는 것	비고
메서드	<code>os.path.abspath(경로)</code>	절대경로 (문자열)	상대경로가 어디를 가리키나

파일 열기

	문법	설명
내장함수	<code>open(경로, 모드, encoding="utf-8")</code>	import 불필요 · 파일 객체를 돌려줌
키워드	<code>with open(...) as f:</code>	블록이 끝나면 자동으로 닫힘 — 실무 기본형
모드	<code>"r"</code>	읽기 (기본값) · 없는 파일이면 오류
모드	<code>"w"</code>	새로 쓰기 — 기존 내용 삭제
모드	<code>"a"</code>	이어 쓰기 — 기존 뒤에 추가
인코딩	<code>encoding="utf-8"</code>	한글이 깨지면 cp949로 바꿔 본다

자주 쓰는 패턴 3가지

외워둘 세 덩어리

```
import os
```

```
# ① 파일을 찾기 전에 위치부터 확인
```

```
print("현재 위치:", os.getcwd())
```

```
# ② 경로를 만들고 존재를 확인한 뒤 연다
```

```
path = os.path.join("data", "08_press.csv")
if os.path.exists(path):
    with open(path, "r", encoding="utf-8") as f:
        for line in f:
            print(line.strip())
else:
    print("파일 없음:", path)
```

```
# ③ 폴더에서 특정 확장자만 골라 경로 만들기
```

```
csv_files = []
for name in os.listdir("data"):
    if name.endswith(".csv"):
        csv_files.append(os.path.join("data", name))
print(csv_files)
```

오류·함정 사전

한 번씩은 반드시 겪는 것들

B-1 · 오늘 실제로 겪은 것

✗ 이렇게 하면	무슨 일이	✓ 바로잡기
<code>ls = [os.listdir()]</code>	리스트가 이중 포장 — len이 항상 1	<code>ls = os.listdir()</code>
<code>len(os.listdir())</code>	폴더까지 세어 "파일 수"가 부정확	<code>[n for n in items if os.path.isfile(n)]</code>
<code>bool(os.path.exists(p))</code>	이미 bool — 중복	<code>os.path.exists(p)</code>
원본 리스트에 덮어쓰기	가공 전 값이 사라져 비교 불가	새 리스트에 담는다
전역 변수에 결과 누적	두 번 호출하면 결과가 겹친다	함수 안에서 만들고 return
<code>os.listdir("Ipynb")</code>	실행 위치에 따라 FileNotFoundError	exists로 확인 후 안내 메시지
파일명에 공백 (…copy.py)	모듈로 import 불가 · 터미널에서 따옴표 필요	공백 없는 이름으로 변경

B-2 · 모듈·경로 함정

✗ 이렇게 하면	무슨 일이	✓ 바로잡기
<code>import math.py</code>	ModuleNotFoundError — 확장자를 붙이면 안 됨	<code>import math</code>
<code>from math import *</code>	이름 충돌 · 출처 가림	필요한 것만 명시적으로
내 파일명을 math.py로	표준 라이브러리와 충돌	다른 이름으로
<code>folder + name</code>	구분자 누락 → data08_press.csv	<code>os.path.join(folder, name)</code>
절대경로 하드코딩	다른 컴퓨터·폴더 이동 시 오류	상대경로 + exists 확인
<code>import pandas</code>	ModuleNotFoundError — 외부 라이브러리 미설치	<code>pip install pandas</code> 후 사용

B-3 · 파일 함정 (오늘 배운 범위)

증상	원인	해결
한글 깨짐	인코딩 불일치	utf-8 ↔ cp949 바꿔 보기

증상	원인	해결
FileNotFoundError	경로가 틀렸거나 작업 디렉토리가 다름	getcwd + exists 로 확인
기록이 매번 사라짐	w 모드는 기존 내용을 지운다	이어 붙일 때는 a 모드
쓴 내용이 저장 안 됨	close를 안 해 임시 공간에만 남음	with open 을 쓴다

개념

오늘 나온 오류의 절반은 **경로 문제**입니다. 그리고 경로 문제는 대부분 **작업 디렉토리를 확인하지 않아서** 생깁니다.

파일이 안 열릴 때 순서는 항상 같습니다 — ① `os.getcwd()`로 지금 어디 서 있는지 확인 → ②

`os.path.abspath(경로)`로 내 상대경로가 실제로 어디를 가리키는지 확인 → ③ `os.listdir()`로 그 폴더에 뭐가 있는지 확인. 이 세 줄이면 원인이 거의 다 드러납니다.

실습 하이라이트 & 다음 시간 예고

데이터 처리 한 사이클의 절반

C-1 · 오늘 자료와 작성 파일

파일	내용	상태
Lecture/08_01_모듈과_경로_다루기	모듈과 import · 표준 라이브러리 · 경로와 os (41p)	전 범위 진도
Lecture/08_02_파일_입출력	open · 모드 · 인코딩 · with (32p 중 도입부)	도입부까지
Python/18_module_1_copy.py	import 세 방식 + datetime	정확 · 파일명 공백
Python/18_module_2.py	os.getcwd · listdir · path.join · exists	작업 디렉토리 의존
Python/08_press.csv	소성가공 프레스 실데이터 (12행)	내일부터 본격 사용
Practice/08_01_example.py	실습 1~6 전부	6/6 제출 · 버그 4건

C-2 · 오늘 배운 것 한눈에

분류	오늘 나온 것	개수
키워드	import · from · as · with	4개
개념	모듈 · 패키지 · 라이브러리 · 절대/상대경로 · 작업 디렉토리 · 인코딩	4개
표준 모듈	math · random · datetime · os	4개
os 기능	getcwd · listdir · path.join · path.exists · isfile · isdir	6개
내장 함수	open · dir · help	3개
파일 모드	r · w · a	3개

C-3 · 데이터 처리 한 사이클

교안 08_02의 마지막 장이 앞으로의 지도를 보여줍니다. 오늘은 **앞쪽 두 칸**을 끝냈습니다.

단계	도구	오늘	비고
① 찾기	os.getcwd · listdir · path.join ·	완료	STEP 4~5

단계	도구	오늘	비고
	<code>exists</code>		
② 열기	<code>open</code> · <code>with</code> · 모드 · 인코딩	완료	STEP 6
③ 읽기	<code>read</code> · <code>readline</code> · <code>readlines</code> · <code>csv.reader</code>	내일	08_02 후반
④ 처리	형변환 · 조건 필터 · 통계	내일	배운 문법 총동원
⑤ 저장	<code>csv.writer</code> · <code>newline=""</code>	내일	08_02 후반

C-4 · 다음 시간 예고

다음 주제	한 줄 미리보기
<code>read</code> · <code>readline</code> · <code>readlines</code>	내용을 꺼내는 세 방법. 끝의 s 한 글자가 결과를 바꾼다 — 문자열이냐 리스트냐
<code>csv.reader</code> / <code>csv.writer</code>	쉼표로 나뉜 표를 행 단위 리스트로. 쓸 때는 <code>newline=""</code> 옵션이 필수
csv 흔한 오류 4종	한글 깨짐 · 빈 줄 끼임 · 계산 오류(문자열 상태) · 첫 줄 헤더 처리
에러 핸들링	강사님 예고 — "파일을 다루다 보면 에러가 먼저 발생할 겁니다". <code>try</code> · <code>except</code>

연결

강사님이 수업 끝에 남긴 말이 내일의 예고입니다 — **"내일 본격적으로 현재 있는 코드를 기반으로 읽고 쓰는 작업을 배우고, 이어서 에러 핸들링까지."** 에러 핸들링을 파일 다음에 배우는 이유도 함께 말씀하셨습니다. **파일은 없을 수도, 깨져 있을 수도, 권한이 없을 수도 있어서** 예외 처리가 가장 절실한 영역이기 때문입니다. 오늘 배운 `os.path.exists`가 그 예방책의 초보 버전인 셈이고, 내일 배울 `try`·`except`가 본격적인 도구입니다.

오늘의 성취와 다음 한 걸음

도구를 만드는 사람에서 고르는 사람으로

오늘 배운 것을 한 문장으로 줄이면 — **필요한 도구를 가져와서, 파일이 어디 있는지 찾아, 열어 본다.** 어제까지 "함수를 만드는 법"을 배웠다면 오늘은 **"이미 만들어진 것을 가져다 쓰는 법"**입니다. 앞으로 배울 pandas·numpy도 전부 이 방식으로 씁니다.

그리고 오늘 처음으로 **실제 설비 데이터**를 만났습니다. 08_press.csv의 마지막 열 상태(0/1)가 우리가 결국 예측하려는 값이고, 진동X·진동Y·전류가 그 단서입니다. 지금은 파일을 여는 것까지만, **이 과정의 목적지가 무엇인지**는 이제 눈에 보입니다.

오늘 헛갈렸던 지점 다시 짚기

헛갈린 것	정리
모듈 vs 패키지 vs 라이브러리	모듈은 파일, 패키지는 폴더, 라이브러리는 통칭. 셋 다 import로 가져온다
import 세 방식 중 뭘 쓰나	기본은 import 모듈명(출처 명확). 자주 쓰는 한두 개는 from import. 관례가 있으면 as
왜 파일을 못 찾나	상대경로는 작업 디렉토리 기준. 코드 파일 위치와 다를 수 있다 → os.getcwd()로 확인
listdir 결과에 폴더가	listdir은 파일과 폴더를 구분하지 않는다 . 파일만 세려면 os.path.isfile로 거른다
왜 join을 쓰나	윈도우 \와 맥·리눅스 / 차이를 자동으로 맞춰 주고 , 구분자 누락 실수를 막는다
w와 a 차이	w는 기존 내용을 지우고 새로 씀. 이어 붙이려면 반드시 a
한글이 깨질 때	인코딩 불일치. utf-8이 기본, 윈도우 엑셀 CSV면 cp949로 바꿔 본다

개념

복습 루틴 추천: ① os.getcwd()를 **다른 폴더에서 각각 실행**해 값이 달라지는 것 확인 → ② os.listdir()에 폴더가 섞인 것을 보고 isfile로 걸러 보기 → ③ 실습 3의 [os.listdir()]에서 대괄호를 빼고 len()이 달라지는지 확인 → ④ 08_press.csv를 with open으로 열어 한 줄씩 출력. ④는 **내일 수업의 예습**입니다.